1. Do we need user accounts/authentication? Is this something to add later?
    a. Supabase
2. "Hosting"
3. Library of datasets for a given community
    a. List view
    b. Card view
    c. Multiple filters
        i. Topic
        ii. Place-identifier
        iii. Different tags
            1. One taxonomy for teachers
                a. Standards-specific
            2. One for community
                a. Topic-specific (I don't know if this is possible to automate well)
4. 2nd Level individual dataset views.
    a. Dataset preview
        i.
5. Different views for type of user (i.e., what tags are visible/available)
    a. Teachers
        i. Both sets of tags
            1. Should be able to filter on either/both
    b. Community
        i. Topical only
    c. Student
        i. Topical only ? Neither?

Documentation to replicate for another community

1. Worksheet of place identifiers (example?)

From my conversation with Claude:

Community Data Libraries: Key Decisions & Technology Stack

Project: Platform for K-12 schools to discover and access educational datasets for their communities
Prepared: December 28, 2024

Core Concept

The Problem:

- Teachers don't know what data exists for their community
- Data is scattered across government agencies
- Technical barriers prevent classroom use

The Solution:

- Central catalog of curated datasets (we maintain)
- Community libraries that filter to local geography (schools/communities deploy)
- No-code setup via web forms (teachers/community members configure)

Key Innovation: Geography-based filtering. One dataset entry ("Census ACS Demographics") becomes 3,000+ unique access points—one for each county that has the data.

Architectural Decisions

1. Distributed, Not Centralized

Decision: Each community owns their own library instance

Why:

- Community ownership and control
- Customization (can add local datasets)
- Resilience (one library failing doesn't affect others)

Implementation:

- Central catalog (one repository, we maintain)
- Template repository (communities clone and deploy)
- Many instances (one per community)

---

2. Static Sites, Not Dynamic Applications

Decision: Build-time generation, not runtime server

Why:

- Free hosting (Vercel/Netlify free tiers)
- Fast performance (pre-generated HTML)
- No database needed (everything in Git)
- Simple deployment (one-click)

Implementation:

- Fetch catalog at build time
- Filter to community geography
- Generate static pages
- Deploy to edge network

---

3. Git-Based Content, Not Database

Decision: Store everything in YAML/JSON files in Git repositories

Why:

- Version control built-in
- No database to maintain
- Easy to review changes
- Transparent and auditable
- Free infrastructure

Implementation:

- Dataset metadata: YAML files
- Community config: YAML file
- Generated catalog: JSON file
- All in Git repos

---

4. Forms Not Code

Decision: Teachers configure via web forms (DecapCMS), never edit YAML directly

Why:

- No technical knowledge required
- Validation prevents errors
- Visual interface (like WordPress)
- Still commits to Git (version control)

Implementation:

- DecapCMS at `/admin` route
- Forms generate YAML files
- Commit triggers rebuild
- No manual file editing needed

---

5. Catalog-Driven Dropdowns

Decision: Extract geographic identifiers from catalog datasets

Why:

- Shows only identifiers that unlock datasets
- Auto-updates as catalog grows
- Dropdown becomes discovery tool

Implementation:

- Script scans catalog metadata
- Extracts used geographic identifiers
- Generates dropdown data files
- Teachers only see valid options

Example: If catalog has a dataset using county-level FIPS codes, one drop-down provides all county-level FIPS code for possible selection by teacher/user

---

Note: I cut #6 :)

7. All 69 Geographic Identifiers

Decision: Support comprehensive geographic coverage, not just basics

Why:

- Different communities need different identifiers
- Tribal schools need tribal codes
- Rural areas use different geography than urban
- Flexibility for all use cases

Implementation:

- Organize into 12 categories (core, census, tribal, etc.)
- Collapsible form sections (not overwhelming)

Categories:

1. Core Geographic (State, County, Place, CDP)
2. Census Geography (Tracts, Block Groups, ZIP, PUMA)
3. Tribal Areas (10 tribal-specific identifiers)
4. Regional/Statistical (CBSA, CSA, Urban Area, MSA)
5. Political/Administrative (Congressional, state, school district)
6. Environmental (Watersheds, EPA regions, parks)
7. Transportation (MPO, transit, DOT, FHWA)
8. Emergency Management (FEMA, fire, EMS)
9. Agriculture/Planning (FSA, extension, COGs)
10. Utility Districts (Electric, water)
11. Specialized (Healthcare, labor, media, climate)
12. Reference (Coordinates, area, historical names)

---

8. Automated Dataset Tagging

Decision: Use Python scripts to compute statistical properties, not manual tagging

Why:

- Saves time
- Consistent and objective
- Teachers can discover by statistical properties
- Can enhance over time

Implementation:

- Download sample data
- Runs: `python tag-dataset.py sample.csv`

- Script outputs tags: "skewed", "contains-outliers", "good-for-correlation"
- Add to dataset YAML file

Tag types:

- Distribution (normal, skewed, bimodal)
- Outliers (present/absent, affects mean)
- Structure (univariate, bivariate, multivariate)
- Size (small <100, medium 100-1000, large 1000+)
- Correlation (strong, moderate, weak)
- Inference tests (suitable for t-test, chi-square, regression, etc.)

Technology Stack

Frontend Framework: Astro

What: Static site generator optimized for content

Why:

- Minimal JavaScript (fast for schools)
- Built for content-heavy sites
- Easy to learn
- Great markdown support
- Can use React components where needed

Alternatives considered:

- Next.js (too heavy, requires more JS)
- Hugo (too limited)
- Jekyll (Ruby dependency, slower builds)

Content Management: DecapCMS

What: Git-based CMS (formerly Netlify CMS)

Why:

- No database needed
- Beautiful admin UI at `/admin`
- Git commits on save (version control)

- Powerful form builder
- Type-ahead dropdowns (relation widgets)
- Free and open source

How it works:

1. Teacher visits `communitydatalibraries.org/admin`
2. Logs in with GitHub OAuth
3. Fills out forms (looks like WordPress)
4. Clicks Save → commits to GitHub
5. Auto-triggers rebuild

Deployment: Vercel or Netlify (offer both)

Why Vercel:

- One-click deploy from template
- Automatic builds on commit
- Free tier: 100GB bandwidth/month
- Edge network (fast globally)
- Serverless functions available

Why Netlify:

- Better DecapCMS integration (Git Gateway)
- Identity service (authentication)
- Form handling built-in
- Also generous free tier

Recommendation: Offer both, slight preference for Netlify if using DecapCMS features heavily

Costs:

- Free for typical school (500 students, ~10GB/month)
- If exceeded: $19-20/month

UI Components: Flowbite

What: Component library built on Tailwind CSS

Why:

- Pre-built components (cards, tables, modals, dropdowns)
- Consistent design system
- Works perfectly with Tailwind
- Responsive out of the box
- Free and open source
- Great documentation
- Includes chart examples (ApexCharts and Chart.js integration)

Use cases:

- Dataset cards for browsing
- Search and filter UI
- Navigation and menus
- Modals for dataset details
- Forms and inputs
- **Data visualization** (chart examples pre-styled)

Chart integration: Flowbite provides ready-to-use examples for:

- ApexCharts (modern, interactive)
- Chart.js (lightweight, simple)

This means charts will automatically match your Flowbite UI design with minimal custom CSS.

Alternative considered:

- DaisyUI (also good, Flowbite chosen for better docs and chart examples)

Data Visualization: Flowbite-Integrated Options

Important: Flowbite (our UI component library) has built-in examples and styling for both ApexCharts and Chart.js. This means easier integration and consistent design.

For Charts/Plots:

Option 1: ApexCharts ⭐ Flowbite has examples

- Modern, interactive charting library
- **Flowbite integration:** Pre-styled examples, design consistency
- Beautiful defaults, professional look
- React integration: react-apexcharts
- Bundle size: ~150KB gzipped
- Extensive chart types (heatmaps, treemaps, etc.)

- Interactive tooltips, zoom, pan
- MIT licensed (free)
- Best for: Polished, modern appearance that matches Flowbite UI

Option 2: Chart.js ⭐ Flowbite has examples

- Most popular charting library
- **Flowbite integration:** Pre-styled examples, design consistency
- Canvas-based rendering
- Bundle size: ~60KB gzipped
- Simple API, good docs
- React wrapper: react-chartjs-2
- Best for: Minimal bundle size while keeping Flowbite design

Option 3: Recharts

- React-native declarative API
- **No Flowbite examples** (would need custom styling)
- Simpler, cleaner code
- Bundle size: ~140KB gzipped
- Built on D3.js
- Educational-focused simplicity
- Composable components (great with Astro)
- Tree-shakeable
- Best for: If you want maximum simplicity and don't mind custom styling

Recommendation: Choose ApexCharts or Chart.js to leverage Flowbite's existing examples and ensure design consistency.

Decision point: Developer evaluates based on:

- **Flowbite integration** (ApexCharts or Chart.js = easier)
- **Same ecosystem** (ApexCharts + ApexGrid = consistent API)
- Visual quality needs
- Performance budget
- Ease of implementation
- Feature requirements

Strong recommendation:

1. **Best pairing:** ApexCharts + ApexGrid (same company, consistent design, both Flowbite-compatible)
2. **Alternative:** Chart.js + simple tables (lightweight, Flowbite-compatible)
3. **Established:** ApexCharts + AG Grid (more mature grid, proven at scale)

Advantages of ApexCharts + ApexGrid:

- Same company = consistent API and design language
- Built to work together
- Single support/documentation source
- Likely better integration between charts and tables
- Both work with Flowbite styling

For Data Tables:

Option 1: ApexGrid ⭐ Same company as ApexCharts

- From the ApexCharts team (perfect pairing)
- Modern data grid built for React
- Seamless integration with ApexCharts
- Sorting, filtering, pagination
- Consistent API and design if using ApexCharts
- Bundle size: ~TBD (newer product)
-
- **Best for:** If using ApexCharts (same ecosystem, consistent design)

Option 2: AG Grid

- Enterprise-grade grid
- Free community edition
- Advanced features: sorting, filtering, pagination
- Bundle size: ~200KB gzipped
- Export to CSV/Excel
- Handles large datasets well
- Most mature option
- **Best for:** Professional, feature-rich tables (independent of chart choice)

Option 3: TanStack Table (React Table)

- Headless table library
- Fully customizable
- Bundle size: ~15KB gzipped
-
- No built-in UI (you style it)
- More work but lighter weight
- **Best for:** Custom designs, performance critical

Option 4: Simple HTML Tables

- No library needed
- Just CSS styling with Tailwind

- Bundle size: 0KB
- Limited features
- **Best for:** Simple data preview only

Decision point: Developer evaluates based on:

- Feature requirements (export, filtering, etc.)
- Bundle size constraints
- Customization needs
- Development time available

Recommendation approach: Start simple (Recharts + simple tables), upgrade to ApexCharts + AG Grid if needed after pilot feedback.

Performance Budget & Optimization Strategy

Performance budget for dataset detail pages:

- Total JS bundle: <500KB gzipped
- Time to interactive: <3 seconds on 3G
- Lighthouse score: >85
- First contentful paint: <1.5 seconds

Bundle size comparison (with Flowbite integration):

- **ApexCharts + ApexGrid** (same ecosystem): ~150KB + ~TBD = TBD (need to verify)
- **ApexCharts + AG Grid** (established): 150KB + 200KB = 350KB
- **Chart.js + Simple Tables** (lightweight): 60KB + 0KB = 60KB
- **Chart.js + AG Grid**: 60KB + 200KB = 260KB

ApexGrid note: Newer product, need to verify actual bundle size. But same company

as ApexCharts = likely optimized for use together.

Strong advantage of ApexCharts + ApexGrid:

- Same company = designed to work together
- Consistent API (learn once, use for both charts and tables)
- Both Flowbite-compatible
- Single source of support/documentation
- Likely shared dependencies = smaller combined bundle than two separate libraries

Recommendation hierarchy:

1. **Best integration:** ApexCharts + ApexGrid (same ecosystem, Flowbite-styled)
2. **Most established:** ApexCharts + AG Grid (proven at scale, 350KB)
3. **Best performance:** Chart.js + Simple Tables (lightweight, 60KB)
4. **Good compromise:** Chart.js + AG Grid (260KB)

Optimization strategies (regardless of choice):

- Use Astro's `client:visible` for lazy loading charts
- Code splitting (charts only on detail pages, not homepage)
-
- Image optimization (WebP format, lazy loading)
- CDN caching (automatic with Vercel/Netlify)
- Tree shaking (only import what you use)

Decision approach:

1. **Start with Flowbite examples** - Use ApexCharts or Chart.js (already integrated)
2. **If using ApexCharts** - Consider ApexGrid for tables (same ecosystem)
3. Build one dataset detail page using Flowbite examples
4. Test with pilot schools
5. Gather feedback on:
    - Is visual quality important? (ApexCharts + ApexGrid wins)
    - Is performance critical? (Chart.js + simple tables wins)
    - Do students engage with previews?
    - Are advanced table features needed? (AG Grid or ApexGrid)
    -

- Choose final combination
- Monitor performance with Lighthouse CI

Key principle: Leverage Flowbite's existing work AND consider ecosystem alignment (ApexCharts + ApexGrid from same company = easier).

Package Manager: pnpm

What: Fast, disk space efficient package manager

Why:

- 3x faster than npm
- Saves disk space (uses content-addressable storage)
- Stricter dependency resolution (prevents phantom dependencies)
- Compatible with npm ecosystem

- Better monorepo support (if needed later)
- Industry adoption growing

Benefits:

- Faster installs in CI/CD (GitHub Actions)
- Smaller node_modules
- More deterministic builds
- Same commands as npm (pnpm install, pnpm run, etc.)

Alternative considered:

- npm (slower, larger disk usage)
- yarn (good but pnpm is faster)

Build Automation: GitHub Actions

What: CI/CD built into GitHub

Why:

- Free for public repos
- Automatic builds on commit
- Can schedule rebuilds (weekly catalog updates)
- Easy to configure

Use cases:

- Build catalog.json when datasets added
- Validate YAML on pull requests
- Rebuild community libraries weekly
- Extract dropdown data from catalog

Data Processing: Python + Node.js

Python for:

- Automated tagging scripts (scipy, pandas)
- Dataset analysis and validation
- Data extraction from APIs

Node.js for:

- Build scripts (YAML → JSON)
- Catalog generation
- Astro build process

Both free, widely supported, good documentation

Hosting Repository: GitHub

Why:

- Free for public repos
- 2,000 Actions minutes/month free
- Familiar to developers
- Good for open source
- Integrates with Vercel/Netlify

Structure:

- Central catalog: One repo (public)
- Template: One repo (public)
- Community libraries: Many repos (each library)

Optional (Phase 2+): Supabase

What: PostgreSQL database + file storage + authentication

When needed:

- Large file uploads (>100MB datasets)
- User accounts across libraries
- Cross-library search
- Analytics dashboard

Why Supabase:

- PostgreSQL (not proprietary)
- Row-level security
- Free tier: 500MB database, 1GB storage
- Real-time subscriptions
- Open source

Phase 1 doesn't need this - everything in Git

Data Sources & Validation

Official Government Sources

Geographic identifiers from:

- U.S. Census Bureau (FIPS codes, coordinates, CBSA)
- NCES (school districts, LEA IDs)
- Bureau of Indian Affairs (tribal codes)

Datasets from:

- Census Bureau (ACS, Decennial, Business Patterns)
- NCES (schools, finance, achievement)
- EPA (air quality, water, superfund)
- CDC (health outcomes, risk factors)
- BLS (employment, wages)
- USDA (agriculture)
- NOAA (climate)
- FBI (crime statistics)

Data Quality

Validation:

- JSON Schema validation for dataset metadata
- Automated checks on build
- Manual review of first datasets
- Curator quality assurance

Updates:

- Catalog: Weekly/monthly additions
- Identifiers: Annual refresh from Census/NCES
- Community libraries: Auto-update via scheduled rebuilds

Security & Privacy

Authentication

Phase 1:

- GitHub OAuth for DecapCMS access
- Or Netlify Identity
- No custom auth needed

Phase 2+ (if needed):

- Supabase auth for cross-library features
- SSO for districts
- Row-level security for private libraries

Access Control

Phase 1:

- Public libraries (all datasets visible)

Phase 2+ (optional):

- Private libraries (password-protected)
- District SSO
- Permissions for community-contributed datasets

Key Workflows

1. Teacher/Community Member Deploys Library

1. Click "Use Template" on GitHub (creates their repo)
2. Click "Deploy to Netlify" button
3. Site builds and deploys (3 minutes)
4. Visit `yourschool.netlify.app`
5. First load redirects to `/admin` setup

2. Teacher/Community Member Configures Library

1. Go to `/admin`
2. Log in with GitHub OAuth
3. Fill out form
4. Click "Publish"

5. DecapCMS commits `config/community.yml`
6. Netlify detects commit → rebuilds site (3 minutes)
7. Site now shows 42 filtered datasets

3. Adding Datasets to Master Catalog

1. Clone catalog repo
2. Copy example dataset YAML
3. Fill in all metadata fields:
    - Basic info (title, description, provider)
    - Geographic coverage (which levels available)
    - Access info (API URLs, keys needed)
    - Teaching metadata (grade levels, subjects, uses)
4. Download sample data for this dataset
5. Run: `python scripts/tag-dataset.py sample.csv`
6. Copy computed tags to YAML
7. Add teaching notes
8. Commit and push to GitHub
9. GitHub Action validates and builds catalog
10. New dataset appears in catalog.json
11. All community libraries get it on next rebuild

4. Student Uses Dataset

1. Visit their school's library site
2. Browse datasets or search
3. Click dataset → see details
4. See pre-filled API URL for their county
5. Click "Download CSV" (serverless function fetches data)
6. Open in Excel/Google Sheets
7. Analyze for class project

Build Process

Central Catalog Build

Trigger: Commit to `datasets/**/*.yml`

Process:

1. GitHub Action runs
2. Load all YAML files
3. Validate against schema
4. Generate `dist/catalog.json`
5. Extract identifiers → `dist/identifiers.json`
6. Commit generated files
7. Success!

Output:

```
{
  "version": "1.0",
  "generated_at": "2025-01-15T10:30:00Z",
  "dataset_count": 75,
  "datasets": [...]
}
```

Community Library Build

Trigger:

- Commit to repo (teacher edits config)
- Scheduled (weekly catalog updates)
- Manual (redeploy button)

Process:

1. Fetch latest `catalog.json` from central repo
2. Load `config/community.yml`
3. Run `filter-catalog.js`:
    - For each dataset in catalog
    - Check if available at teacher's geography
    - If yes: add to filtered list, pre-fill API URLs
4. Save `src/data/available-datasets.json`
5. Astro builds pages using filtered data
6. Deploy to Vercel/Netlify
7. Live in 2-3 minutes!

Filtering Logic Example

Teacher's config:

```
community_identifiers:
  state_fips: "17"
  county_fips: "17167"
  school_district_lea: "1709930"
  zip_codes: ["62701", "62704", "62711"]
```

Dataset in catalog:

```
id: "census-acs-demographics"
geographic_coverage:
  available_levels:
    fips_code: true      # ✅ County level
    zip_code: false      # ❌ Not at ZIP level
    school_district: false
```

Filtering logic:

```
// Does dataset match teacher's geography?
const matches =
  (dataset.available_levels.fips_code && teacher.county_fips) ||
  (dataset.available_levels.zip_code && teacher.zip_codes.length > 0) ||
  (dataset.available_levels.school_district && teacher.school_district_lea);

if (matches) {
  // Pre-fill API URL
  dataset.api_url = dataset.url_template.county
    .replace('{fips}', teacher.county_fips);

  availableDatasets.push(dataset);
}
```

Result: Teacher sees this dataset because it's available at county level and they provided county FIPS.

Research Foundations

Digital Libraries (JCDL):

- Automatic tag recommendation (topic modeling + hybrid AI/human)
- Teachers want pedagogical metadata, not just technical specs
- Participatory design with communities

CS Education (SIGCSE):

- Data science should be interdisciplinary (not just CS)
- Tools shape what students learn (need age-appropriate interfaces)
- Students need authentic, local data (not national abstractions)

Data Governance (CARE Principles):

- Collective Benefit (communities derive benefit)
- Authority to Control (communities control their data)
- Responsibility (support self-determination)
- Ethics (community wellbeing primary)

Combines:

- FAIR (technical sharing - Findable, Accessible, Interoperable, Reusable)
- CARE (people and purpose - Indigenous data sovereignty)

Success Metrics

Technical Success

- [ ] Catalog builds automatically
- [ ] Template deploys in <3 minutes
- [ ] Filtering accuracy >95%
- [ ] Site loads in <2 seconds
- [ ] Build success rate >98%

Content Success

- [ ] 50-75 datasets by end of spring
- [ ] All datasets have complete metadata
- [ ] Automated tags on all datasets
- [ ] Teaching notes on 30+ datasets

User Success

- [ ] 2-3 pilot schools deployed
- [ ] Teachers configure in <15 minutes
- [ ] No critical bugs reported
- [ ] Positive feedback (>4/5 rating)
- [ ] Teachers using datasets in class

Long-term Success

- [ ] Platform requires <5 hours/week maintenance
- [ ] Community contributions working
- [ ] 10-20 active libraries (by summer)
- [ ] 100+ libraries (Year 2 goal)

What We're NOT Building (Explicitly Deferred)

Phase 1 (Spring)

- ❌ Real-time features
- ❌ User accounts/authentication (beyond GitHub OAuth for admin)
- ❌ Database backend (Supabase deferred to Phase 2+)
- ❌ Large file storage (>100MB)
- ❌ Mobile app
- ❌ Advanced analytics dashboard
- ❌ Cross-library search (deferred)
- ❌ Live API proxying (teachers get direct links)

Can Reconsider Later

If pilots need it, or summer work continues, can add:

- Supabase for large files
- Cross-library discovery
- Advanced filtering
- Analytics
- Community contribution workflow (beyond basic)

Key Decisions Summary

1. **Distributed architecture** - Each school owns their instance

2. **Static sites** - Build-time generation, not runtime
3. **Git-based content** - Everything in version control
4. **Forms not code** - Teachers use DecapCMS, never edit YAML
5. **Catalog-driven dropdowns** - Extract from real datasets
6. **Two-person team** - Developer + curator working in parallel
7. **All 69 identifiers** - Comprehensive geographic coverage
8. **Automated tagging** - Python scripts compute statistical properties
9. **Astro + DecapCMS + Vercel/Netlify** - Modern, free, fast
10. **Government data sources** - Official, validated, free

Technology Summary

Stack:

- Frontend: Astro
- CMS: DecapCMS
- Deploy: Vercel or Netlify
- CI/CD: GitHub Actions
- Languages: JavaScript, Python
- Storage: Git (GitHub)
- Optional: Supabase (later)

All free for typical school usage!

This document captures the core decisions and stack. No timelines, just the "what" and "why" of our technical choices.